Contents

1 Evaluation Framework **2**
1.1 Approach . . . 2
1.2 Test inventory . . . 2
1.3 Why these five tests . . . 2
1.4 ML Test Score . . . 3
1.5 Reproducibility tests . . . 4
1.6 Delivery throughput (DORA) . . . 4
1.7 Drift monitoring counts . . . 5
1.8 Maturity-model positioning . . . 5
1.9 Threats to validity (preview) . . . 6
1.10 What is not tested and why . . . 6

Chapter 1

Evaluation Framework

1.1 Approach

- One paragraph framing: this chapter describes the system-side tests; modelling-side evaluation (MAE/RMSE/sMAPE/rMAE) lives in Ch. **??**.
- Cite single-case study methodology [6]: one platform, evaluated in its real context, no controlled comparison.
- State the honesty boundary up front: numbers are indicative over the study window, not long-run production claims.
- Distinguish two kinds of measurement (introduced in Table 1.1): *objective measurements* (reproducible scripts) and a *structured self-assessment* (the ML Test Score). This distinction matters for the validity discussion.

Table 1.1: Two kinds of test used in this work.

Type	**What it produces**	**Auditable how?**
Objective measurement	A number computed by a script over code, git history, or the database	Re-run the script — same inputs, same number
Structured self-assessment	A rubric score with one-line evidence per item	Read the evidence column; agree or contest each item

1.2 Test inventory

- One sentence: five tests, four objective and one self-assessment, listed in Table 1.2.

1.3 Why these five tests

- Open: Zarour's SLR [7] catalogues nine MLOps practices but produces no quantitative aggregate; we compose tests that each address a measurable face of one or more practices.
- Argue: between them, the five tests evidence eight of Zarour's nine practices (Table 1.3); the ninth (P5 Collaboration) is qualitative by nature and is addressed in the System Implementation chapter, not measured here.

Table 1.2: Tests applied to the system in this work.

Test	Type	Source / instrument	Cite
ML Test Score	Self-assessment	28-item rubric, scored 0 / 0.5 / 1, min-of-sections	[1]
Provenance coverage	Objective	SQL aggregate over `algo.experiment_runs`	[4]
Re-run demonstration	Objective	`scripts/report/reproduce_run.py` — re-execute logged run, compare to stored prediction	[4]
DORA delivery throughput	Objective	`gh`/git mining of `cd_deploy.yml` history	[2]
Drift monitoring counts	Objective	SQL aggregate over `algo.drift_reports`	—

Table 1.3: Mapping of tests to the nine MLOps practices in Zarour et al. [7].

Zarour practice	Tested by
P1 Automating CI/CD	ML Test Score (Infra), DORA throughput
P2 Orchestrating workflows	ML Test Score (Infra)
P3 Ensuring reproducibility	ML Test Score (Infra), Provenance coverage, Re-run demonstration
P4 Versioning	ML Test Score (Data, Infra), Provenance coverage
P5 Collaboration and communication	*not measured here — qualitative, see Ch.* **??**
P6 Continuously train and evaluate	ML Test Score (Model)
P7 Track and log ML metadata	ML Test Score (Model), Provenance coverage
P8 Continuous monitoring	ML Test Score (Monitoring), Drift counts
P9 Implementing feedback loops	Drift counts (re-framed as monitoring; see §**??**)

1.4 ML Test Score

- Rubric origin [1]: 28 tests in four sections (Data / Model development / ML infrastructure / Monitoring); per-item score from $\{0, 0.5, 1\}$.
- Final score: min(section sums). Justify the min-rule (one sentence): the four sections protect against different failure modes, are not substitutable, so the weakest section bounds production-readiness.
- How conducted: a structured self-assessment — every item walked against the codebase, one-line evidence recorded, scores assigned by the explicit rule in Table 1.4. Per-item evidence is committed to `Report/metrics/ml_test_score.md`.

Table 1.4: Scoring rule applied to each of the 28 items.

Score	Meaning
1.0	Capability exists and is automated / tested / repeatable (runs in CI, runs on schedule, has a test)
0.5	Capability exists but is manual, partial, or ad-hoc
0.0	Not done, or genuinely not applicable to the domain

Table 1.5: The four sections of the rubric and what each is designed to detect.

Section (7 items each)	**Failure mode it guards against**
Data tests	Bad inputs reach training / serving (schema drift, missing features, costly features)
Model development tests	Models ship that are worse than a baseline, untuned, or untested on slices
ML infrastructure tests	Pipeline can't be reproduced, debugged, rolled back, or safely canary-deployed
Monitoring tests	Production failure goes undetected (silent drift, stale model, regressed accuracy)

- Honesty rule applied: no point awarded without written evidence; n/a items scored 0 (not awarded points).
- Validity caveat to flag: this is a self-administered rubric — the threats-to-validity section names this and explains the mitigations.

1.5 Reproducibility tests

- Framing: reproducibility is measured in two senses — coverage and demonstration — per [4].

Table 1.6: Two reproducibility tests.

Test	**Definition**
Provenance coverage	% of live runs in `algo.experiment_runs` that carry the complete triple: `git_sha`, (`data_start_utc`, `data_end_utc`), `model_artifact_id`
Re-run demonstration	Take one logged run; using only its committed config + target date, re-execute `LiveRunner`; report the max absolute difference vs the stored prediction

- Why both: coverage is a system-wide property (does every run carry enough metadata to be reproduced?); demonstration is a worked existence proof (here is one logged run, regenerated).
- Why LEAR for the demonstration: deterministic given fixed data; DNN / Transformer add seed / ensemble stochasticity — to be addressed as future work.
- ACM Artifact Badge mapping: claim *Available, Functional, Reusable* by self-review; do not claim *Reproduced* (requires external reviewer).

1.6 Delivery throughput (DORA)

- Definitions: one deploy = one successful `cd_deploy.yml` run on `main` [2].
- Lead time: wall-clock interval from merge commit to `cd_deploy` run start (the interval under the platform's automated control).

Table 1.7: DORA: what is measured, what is excluded, and why.

Metric	Reported?	Why / why not
Deployment frequency	Yes	Mineable from git/CI history alone [2]
Lead time for changes	Yes	Same; throughput metric
Change failure rate	*No*	Requires an incident corpus the study window does not provide
Mean time to restore	*No*	Same: needs labelled production incidents
DORA tier placement	*No*	Tier thresholds are re-clustered yearly; unsafe to assign on a short window

1.7 Drift monitoring counts

- Definition: counts of (a) drift reports written and (b) reports flagged `drift_detected = 1`, over the window, from `algo.drift_reports`.
- Why these and not drift-to-retrain latency: under the reframing argued in the Results chapter (§**??**), drift detection serves monitoring, not triggering. Latency to retrain is therefore not a meaningful metric for this system.
- Boundary condition (to surface in the results chapter): this reframing is conditional on retraining staying cheap; it would flip for expensive multi-year retraining [5].

1.8 Maturity-model positioning

- Maturity-model placement is reported in the qualitative practice-by-practice table in System Implementation (Ch. **??**, Table **??**) — this section justifies the *choice* of model.
- Zarour [7] Table 7 lists five MLOps maturity models (Table 1.8); we position the system against the Google 3-level model.

Table 1.8: The five MLOps maturity models reviewed by Zarour et al. [7], Table 7. All derive from CMM/CMMI.

Model	Levels	Top level
Google	3 (0–2)	CI/CD pipeline automation
Microsoft	5 (0–4)	Full MLOps automated operations
GigaOm	5 (0–4)	Comprehensive MLOps pipeline for frequent model updates
John, Olsson & Bosch (2021)	4 (0–3)	Fully-automated model monitoring
Mateo-Casalí et al. (2022)	5 (0–4)	Optimization (pure CMMI mapping)

- Justify Google in three bullets:
  - Granularity matches the single-host MVP scope; 3 levels avoids over-claiming distinctions a single deployment target cannot meaningfully differentiate (e.g. Microsoft Levels 2–4).
  - Top level (CI/CD pipeline automation) is plausibly attainable for this platform; Microsoft Level 4 ("Full MLOps automated operations") would require multi-environment claims the platform does not make.

  - Open and widely cited industry whitepaper [3]; the most-cited maturity reference in the Zarour SLR.

- Disclose limitation: all five models are descriptive ordinals — they support *positioning*, not *measurement*. The instruments in §1.4–§1.7 carry the measurement load.

1.9 Threats to validity (preview)

- Detailed discussion lives at §**??**; here, one sentence per threat type so the reader sees what we will defend against.

- Frame per [6].

Table 1.9: Threats to validity at a glance; each is discussed under §**??**.

Threat	**Source in this work**	**Mitigation**
Construct	ML Test Score is self-administered	Per-item evidence in `ml_test_score.md`
Internal	Single-system observation; drift conclusion conditional	State the boundary condition; objective measurements carry main load
External	Single-case design	Analytic generalisation only; no statistical claim about MLOps platforms
Reliability	Would another evaluator score the same?	Explicit scoring rules + recorded evidence per item

1.10 What is not tested and why

- Explicit non-claims — writing these down prevents accidental over-reading of the results.

Table 1.10: Explicit non-claims of this evaluation framework.

Not claimed	**Why**
Long-run uptime / availability	Prometheus tier is only days old; passive uptime over a short window is uninformative
Change failure rate / MTTR	No incident corpus in the window (see §1.6)
DORA tier placement	Tier thresholds re-clustered yearly; unsafe on a short window
Comparison vs another MLOps platform	Single-case design; no controlled comparison
Long-run forecasting accuracy	Belongs to the modelling-side chapter (Ch. **??**), not this framework
Statistical generalisation	Single case; only analytic generalisation per [6]

Bibliography

[1] Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. The ML test score: A rubric for ML production readiness and technical debt reduction. In *2017 IEEE International Conference on Big Data (Big Data)*, pages 1123–1132, 2017.

[2] Nicole Forsgren, Jez Humble, and Gene Kim. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

[3] Google Cloud. MLOps: Continuous delivery and automation pipelines in machine learning, 2024. Google Cloud Architecture Center whitepaper. Defines a three-level MLOps maturity model (Level 0 manual, Level 1 ML pipeline automation, Level 2 CI/CD pipeline automation).

[4] Odd Erik Gundersen and Sigbjørn Kjensmo. State of the art: Reproducibility in artificial intelligence. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[5] Ananth Mahadevan and Michael Mathioudakis. Cost-aware retraining for machine learning. *Knowledge-Based Systems*, 293:111610, 2024.

[6] Per Runeson and Martin Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009.

[7] Mohammad Zarour, Hussein Alzabut, and Khalid T. Al-Sarayreh. MLOps best practices, challenges and maturity models: A systematic literature review. *Information and Software Technology*, 183:107733, 2025.